J. Parallel Distrib. Comput. 84 (2015) 87–93

Contents lists available at ScienceDirect

J. Parallel Distrib. Comput.

journal homepage: www.elsevier.com/locate/jpdc

GPU enabled XDraw viewshed analysis

Aran J. Cauchi-Saunders *, Ian J. Lewis

School of Engineering and ICT, University of Tasmania, Private Bag 87, Hobart, TAS 7001, Australia

HIGHLIGHTS

- Performed performance experiments on four different viewshed analysis algorithms across CPU and GPU domains.
- Utilized the C++ AMP framework to ensure cross-platform generalizability.
- Optimized the XDraw viewshed analysis algorithm for efficiency on a GPU and compared to previous algorithms.
- Optimized GPU XDraw algorithm performed well when compared to similar GPU and CPU algorithms.

ARTICLE INFO

Article history:
Received 17 July 2014
Received in revised form
25 May 2015
Accepted 7 July 2015
Available online 26 July 2015

Keywords:
Visibility
GPGPU
XDraw
C++ AMP
Viewshed
Digital terrain visibility

ABSTRACT

Viewshed analysis is an important tool in the study of digital terrain visibility. Current methods rely on the CPU performing computations to linearly calculate visibility for a given position on a portion of digital terrain. The viewshed analysis process can be sped up through the use of a GPU to parallelize the visibility algorithms. This paper presents a novel conversion of the XDraw viewshed analysis algorithm to a parallel context in an effort to increase the speed at which a viewshed can be rendered. The algorithm executed faster than current linear methods and responded well to parallelization. We conclude that XDraw is applicable for GIS applications when rendered in a parallel context.

© 2015 Elsevier Inc. All rights reserved.

1. Introduction

Viewshed analysis is the process whereby terrain is analyzed and found to be visible or invisible for an observer at a given location [2]. A viewshed is a particular region of terrain that is visible to an observer placed on the terrain at any given point [13]. Fundamentally, the purpose of a viewshed is to gain an accurate representation of what is visible for an *observer* at an arbitrary point of some terrain.

An observer can be described as a theoretical entity placed in a digital environment whose purpose is to observe the terrain's states of visibility. For example, consider an observer standing on the floor of a valley surrounded by hills. The observer would have a viewshed that included the entire valley up to the surrounding hills. Any point beyond the peaks of the hills would be invisible to the observer, thus being excluded from their viewshed. Terrain is typically represented by a two-dimensional grid of discrete floating-point values, which indicate the elevation of a portion of topography [14]. This grid is hereafter referred to as a Digital Elevation Model (DEM) or as a dataset.

1.1. Viewshed analysis

Viewshed analysis is the process whereby visibility for a DEM is calculated. Typically, viewshed analysis utilizes Line of Sight (LoS) algorithms such as the R3 and R2 algorithms proposed by Franklin, Ray and Mehta [5] along with the DDA algorithm proposed by Kaučič and Zalik [7]. These three algorithms utilize LoS calculations by projecting rays originating from the observer toward the boundary of the DEM. From this, it can be determined linearly which points are obstructed and which are considered theoretically visible, essentially determining a LoS for each point. The efficiency of these algorithms can be stated as $O(r^2)$ where r is the average radius of the DEM from an arbitrary position in DEM space. As DEMs can be of considerable size, efficiency is a primary concern [14].

1.2. GPGPU

While viewshed analysis can be accurately and efficiently calculated in a linear fashion on a CPU, there exist several

* Corresponding author.
E-mail addresses: ajcauchi@utas.edu.au (A.J. Cauchi-Saunders), ian.j.lewis@utas.edu.au (I.J. Lewis).

http://dx.doi.org/10.1016/j.jpdc.2015.07.001
0743-7315/© 2015 Elsevier Inc. All rights reserved.

precedents of increases in speeds when viewsheds are generated via a GPU; several existing experiments have been performed whereby GPUs are utilized to generate viewsheds more efficiently in some cases. The precedent to perform viewshed analysis on a GPU has been well explored. Chao et al. [1] described a viewshed analysis technique whereby shadow mapping was used to overlay a viewshed on a DEM as if rendering terrain while Zhao, Padmanabhan and Wang [16] also devised an efficient modified R3 algorithm which executed on the GPU [16].

These factors allow each vertex to be computed via an individual instance of a GPU kernel. Furthermore, each octant of the algorithm can be calculated independently of one another, allowing greater spatial independence, thus greater parallelizability. Each thread in a GPU block, represents a single LoS calculation; therefore 16,000 GPU threads for example would represent 16,000 individual parallel LoS calculations, disregarding certain GPU bandwidth limits. This volume of threads fits well with the GPGPU paradigm, which argues for a very high number of independent threaded operations being executed simultaneously over a sustained [10,14]. There has been a concerted effort to discover the potential performance benefits of using the GPU as a viewshed processor [15,12,3], which aims to either modify existing CPU algorithms, or design new algorithms specifically for CUDA hardware; [6] presents a novel algorithm for 'combing' the DEM via thread directions. Whilst the algorithm gained notable performance increases, the speedup relied on a CUDA specific design, particularly optimizing for CUDA warps. Whilst platform specific optimizations can yield significant gains in performance, it is important to note that as the solution narrows to specific hardware, the real world generalizability of the algorithm may suffer. CUDA specific optimizations also apply to the work of Stojanovic and Stojanovic [11].

1.3. XDraw

XDraw is a viewshed analysis algorithm developed by Franklin, Ray and Mehta [5]. Xdraw has hitherto been executed only in GPU contexts. The following sections will describe the fundamentals of Xdraw and an attempt to parallelize it. XDraw conceptually divides the DEM via the cardinal directions of North, East, South and West along with the intermediate North-East, North-West, South-East and South-West compass directions. Therefore, the DEM is divided into eight 45° octants such that the lines of demarcation between the octants are considered as following the eight points of the compass.

The primary difference between XDraw and similar viewshed analysis algorithms, and also XDraw's unique strength, is that the order in which these calculations are performed is fundamental. Instead of calculating a LoS to any given point on the terrain and then progressing to that point's neighbor, XDraw assumes a regular growth pattern emanating from the observer's position on the DEM in which LoS information is gathered. Along each of the compass direction lines from the observer to the DEM boundaries, exact LoS calculations are made due to the grid like structure of the DEM. For points falling between the eight compass directions, an approximation of LoS information is used. For each vertex v_t on the currently tested ring i, only the two LoS values stored in vertices v_a and v_b which reside in the next inner ring $i-1$ are queried. Both v_a and v_b straddle the ray projected from the observer to v_t and are the only two pieces of data needed to calculate an approximate LoS for v_t. An important consideration of XDraw is that its approximate nature for generating the octant vertices' LoS compounds upon itself as the rings grow further from the observer. Franklin, Ray and Mehta [5] attempt to alleviate this problem by implementing an approximation of v_t, by combining the mathematical maximum and minimum of the LoS values stored at v_a and v_b, this ensures a reasonable level of accuracy for each approximate vertex calculated. Fig. 1 displays the growth of an XDraw calculation.

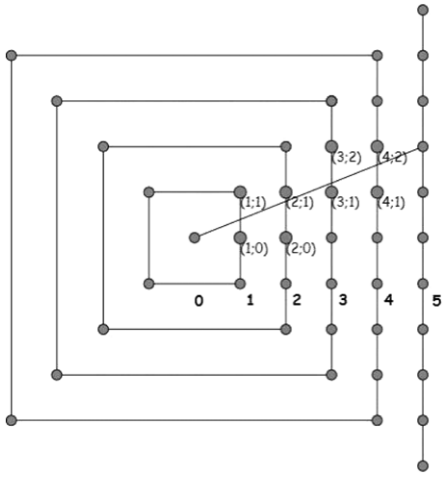


Fig. 1. XDraw growth pattern.

Whilst the algorithms noted in the previous section perform well when compared to traditional CPU computations of viewsheds, a hitherto unexplored opportunity exists to parallelize the XDraw algorithm. The spatial independency characteristics for XDraw first allow a high level of parallelizability to be attained. As each XDraw ring grows in size, each vertex can be calculated independently of its neighbors on the same ring. For more information concerning the exact workings of XDraw, we refer the reader to the work of Franklin and Mehta [5].

1.4. Algorithmic conversion of XDraw

XDraw relies on an expanding ring of visibility operations to calculate a viewshed. To fully parallelize the algorithm, several limiting factors of the algorithm must be observed.

First, the growth rings must be synchronized, as each point of visibility relies on the preceding two point's visibility calculations. Second, each octant can be fully decoupled from its neighbors' visibility calculations along the adjacent edge of the right-angle. Recall that the fundamental algorithmic components of XDraw consist of eight 45° octants that are fully independent of one another. Each octant represents one-eighth of the terrain, in the form of a right-angled triangle. Fig. 1 displays two octants, that of ENE and ESE. Independence refers to any calculation that occurs in one octant are decoupled for any other point calculations in the seven other octants. Recall also that each ring of XDraw calculations is independent from its neighbor and can be safely calculated in isolation.

The algorithmic conversion process to be utilized in this paper exploited these characteristics. For each LoS calculation on any given ring, an individual thread can be assigned to a discrete LoS calculation. A ring, refers to a contiguous square of points, currently being calculated. This ring expands outwards to the next square ring of points, once all calculations in this square ring are complete. It can be then stated that for each point v on ring i, the visibility of v is calculated by using the height of the two inner two points v_a and v_b of the ring at positions $i-1$. For example, referring to Fig. 1, the point at position (4, 2) can be calculated by observing the height of points (3, 2) and (3, 1). If the heights of v_a and v_b obscure v, the visibility of v is deemed as low or invisible. Via this process, the pool of threads that can be simultaneously executed by the SMs on the GPU at any point are the following:

$$2n + 2m - 4$$

where n and m represent the length and width of the DEM. Once an entire ring has been calculated by the GPU, the following ring $i + 1$ is then calculated simultaneously in the same fashion.

The overall number of threads for XDraw, assuming an entirely central located observer and a DEM dimensional relationship where $n = m$, is:

$$n \times m.$$

This process can be seen to generate a high number of kernel threads for the subsequent computations.

The graphics processing unit is incapable of dealing with moderate levels of branching in an efficient manner, thus reducing the potential efficiency of the algorithm [13]. Furthermore, the flow of execution on a GPU calculates each potential path from a potential code branch and performs conditional memory writes for all data. This model is clearly inconsistent with the GPGPU paradigm of arithmetic intensity and spatial independency [9]. Specifically, XDraw maintains a fundamental branching characteristic whereby the elevation of the previous points are compared to the elevation of the currently sampled point in an attempt to determine the LoS for said point. By converting the LoS calculations from a branch structure to a comparison operation, theoretical efficiencies can be gained.

Specifically, when the LoS comparison is effecting, rather than executing a conditional statement, the maximum value between the current point and previous point is calculated. This value is then automatically stored in the resultant array. As such, no branching operations are made.

Several optimizations were considered which could potentially yield performance increases when compared to a CPU rendition of XDraw. These can be split into two categories, general I/O optimizations, and specific data-tiling optimizations. In the general case, to prevent unnecessary data read and write times, the initializing input data containers for XDraw (line of sight and elevation arrays) were explicitly nulled by the GPU kernel such that only a pointer to these arrays was copied to GPU memory, rather than the entire array. Recall that the input elevation array could be composed of millions of elevations, such that would largely slow the execution of the processing pipeline. In a similar fashion, when the GPU kernels were destructed, only the resultant visibility array is passed back to host memory and the two previously mentioned input arrays are explicitly dereferenced by the GPU and not returned. The following fundamental I/O operations compose the XDraw algorithm and have been identified as performance bottlenecks.

Through testing, it was determined that several data-tiling optimizations were possible, with respect to the unique structure of the XDraw algorithm. It was determined that to calculate the visibility of a points, the algorithm requires two global reads for all v_a and v_b LoS information, one global read for elevation of i and finally a global write of the visibility result. These bottlenecks have been determined as the points at which noticeable delays occur in the exaction of the algorithm during testing. While the final global write is unavoidable when the kernel finishes processing, the three read operations from global memory can be optimized by making use of the underlying cache memory for each stream multi-processor.

To this end, a tiled memory management pattern has been devised to reduce the impact of global I/O on the performance of the XDraw algorithm. Fundamentally, each XDraw calculation requires three values stored in global memory. These values can be visualized as a two-dimensional array representing the growth ring i and the inner ring $i - 1$. As this calculation octant expands, values are pre-loaded into the buffer for upcoming LoS calculations. Each tile is the current width of one octant and all values necessary for this number of calculations are directly transferred into the buffer at once. For example, point $(4, 2)$ in Fig. 1 would sit in a buffer five floats wide. As the global memory I/O can transport a large number of values in one transaction, this process is seen as more-cost effective when performed in larger batches.

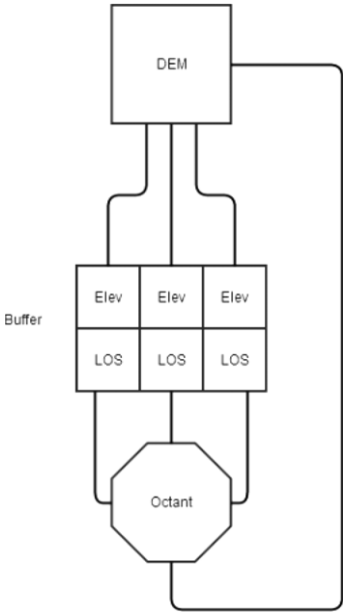


Fig. 2. XDraw-O tiled memory.

Specifically for XDraw Optimized (XDraw-O), as each ring grows for each octant, the elevation and line of sight data is transferred from global memory to an intermediary buffer which exists in GPU static memory, that is, a cache for each stream multi-processor (see Fig. 2). As this data is passed through at once, once the octant begins to calculate, the data I/O times are largely reduced as the SM does not need to stall for a global I/O operation. This process is repeated for each octant across each ring of calculation until the DEM boundary is hit. The size of the octant buffer also increases as the octant width grows. The actual data-structure is composed of a two-dimensional array, with one axis as the calculated distance from the observer to the edge of the DEM at compile-time, by 2 cells as the other axis. This size can scale beyond the hardware's capabilities to a size larger than any DEM tested and this limit will not be hit unless comparatively larger DEMs are utilized. In that occurrence, the DEM size may exceed the global GPU memory size, negating this limitation. The major contribution to the performance of XDraw-O is the tiled memory-managed solution. This allows for efficient storage of relevant visibility calculation variables in the SM's registers as the growth-ring proceeds in lock-step outwards from the observer.

A further optimization applied to XDraw, involved changing the order in which the octants are processed. For XDraw, the individual octant calculations were split into eight separate kernel paths which were then encompassed into an over-arching loop that ensured the rings of growth were full synchronized between the octants. This process was considered for XDraw, as it was easier to determine which GPU thread processed any portion of a given octant. For XDraw-O, each octant is now contained in one of two separate kernel solutions: Kernel 1, which processes the NNE, NNW, SSE and SSW octants and Kernel 2 which processes the ENE, WNW, WSW and ESE octants. Splitting the calculations into two separate kernel solutions was the result of experimentation with the kernel structure of the algorithm to reduce the number of code branches when executing each octant's visibility calculation. Each kernel solution now only processes the relevant octants in its queue, such that only one half of the ring is calculated by

Table 1
Description of computational viewer points (test points).

Test points	Description
Test Point 1	In a moderately constrained region, to generate a reasonably large viewshed and to also ensure that the visibility of the algorithm is correctly constrained at medium distances.
Test Point 2	In a deeper valley with reduced visibility, with the aim to produce a relatively small viewshed and to also test how the algorithm constrains visibility at closer distances.
Test Point 3	On top of the highest point of the topography.

Table 2
DEM sampling data.

	DEM500	DEM300	DEM250	DEM200	DEM150
Discrete points	818,055	2,272,375	3,268,573	5,106,006	9,077,344
Sample rate	500 m^2	300 m^2	250 m^2	200 m^2	150 m^2

each kernel solution. This effort increased the lines of code in the program, but reduced the effort required by the GPU to make decisions concerning when the octant had been fully rendered to the boundaries. Future efforts may include a greater number of kernel versions. This model provides a compromise between kernel complexity and determining which path each GPU thread should follow. It was determined that changing the structure of XDraw's ring processing may produce an increase in performance over XDraw. Each octant maintains its own static buffer of memory, which is filled as the calculation ring grows further outwards.

2. Method

Several experiments were devised to determine the efficiency of XDraw when converted to a GPU context. Each algorithm was executed as a CPU based program extending the EonFusion GIS software. EonFusion is a 4D GIS suite which allows third-party modular extensions to operate on a DEM [8]. The algorithms processed each dataset five times to ensure replicable results to ensure a wide spread of viewsheds are maintained. This follows the precedent set by [16]. Additionally, the observer point was placed in three separate locations across the five DEMs (see Tables 1 and 2).

In addition to each algorithm executing three viewsheds with disparate observer locations, each algorithm was executed across a varying degree of DEM sizes. The five following datasets were used as the processing input for the viewshed algorithms. The DEM sizes correspond generally to typical raster-based DEMs used in GIS contexts. The sizes of the raster DEMs, expressed as point sampling rates, widths and heights were as follows.

Each DEM represents the topography of the island of Tasmania, Australia and the original DEM was specifically generated via the process of aerial photogrammetry. To produce an improved algorithm for generating a viewshed as efficient as or more so than current efforts, the XDraw algorithm was executed in a GPU environment to aid in the parallelization of the visibility calculation. It was hypothesized that due to the increased parallelism of the GPU, the rendering of a viewshed could be hastened due to the implicit characteristics of XDraw being congruent with the GPGPU paradigm. Furthermore, as key research into GPU parallel computing continues, a trend toward vendor-specific frameworks such as NVIDIA's CUDA framework has become more prevalent. This paper attempts to create a GPU-agnostic solution to the GPU viewshed analysis problem by utilizing Microsoft's C++ AMP framework, a superset of DirectX's DirectCompute API. This means that XDraw can execute on NVIDIA, AMD and Intel graphics cards, which support DirectX 11.

Several variants of popular visibility algorithms were compared in an attempt to determine the performance between GPU and GPU optimized execution speeds. This comparison occurred in three distinct stages. CPU tests were performed to determine a baseline performance measure. GPU tests were the subsequently executed to determine raw speed increases when the CPU algorithms were transposed to a GPU context. Finally, XDraw was extended to fully support GPU tiling, general branching and performance optimizations. The non-XDraw algorithms tested in this paper, R3, R2 and DDA were transposed from a CPU context to a GPU context with little modification to the underlying structures and control flow. For further comprehension on the specific details of the sightline algorithms, we refer the reader to Kaučič and Zalik [7].

The tests were performed on an AMD 6870 GPU paired with an Intel i5-3570 with 12 GB of DDR3 RAM. All algorithms on the CPU and GPU were executed five times and were averaged to reduce differences in load on the operating system, CPU or GPU. The algorithms were implemented in C++ AMP and the results were passed back to a C# driver module.

Results on accuracy and efficiency were at that point recorded. Performance data was captured as the time difference between the algorithm beginning to process the raw data and the moment when the algorithm passed the completed viewshed back to the host program. This means that no performance data was captured for transferring the DEMs to and from the GPU, which can be considered as beyond the scope of this study.

3. Results and discussion

The following figures listed in Table 3 details the average algorithmic performance of XDraw-O, when compared to the four previous algorithm execution times. Along with this, the following graph listed in Fig. 2 displays the performance characteristics of XDraw against XDraw-O.

It can be observed from the results presented in Table 3 that XDraw-O averaged an execution speed of 0.0375 s across all 15 tests cases. This is compared to XDraw which executed for an average of 0.133 s on average. This represents an average of a 72.2% reduction in computation time when averaged across the five DEMs and three test points. This represents a large decrease in execution time across the three test cases over the five DEMs. The CPU results indicate that in all cases, the GPU versions of the viewshed analysis algorithms performed faster equivalent executions of the viewshed analysis. This result corresponds well with the literature. Surprisingly, the raw bandwidth increase of the GPU provided a large decrease in execution time for GPU algorithms, regardless of fully considered and optimized memory patterns. For each of the algorithms, the conversion from CPU to GPU yielded significant performance improvements. DDA, R3, R2 and XDraw all executed faster on average on a GPU than on the CPU as noted in Table 4. This performance increase may be due to the fact that line of sight calculations fit well into the multiple thread paradigm of GPGPU computing. Indeed, this result has been sorted in the literature, especially by Zhao, Padmanabhan and Wang [16]. The increase in individual algorithmic performance for each DEM does not follow a standard pattern. The increase in execution speed for R3 on CPU and GPU for DEM500 was nearly a factor of ten. This may be due to the fact that greater number of calculations can be performed relative to the size of the DEM, without needing to perform I/O operations. This effect becomes less pronounced as DEM size grows, and the GPU must perform more data reads and writes relative to the size of the DEM as it grows, decreasing the speed improvements. As memory management is critical for effective GPU operations, the effect of the size of a DEM in GPU memory plays a more significant role in execution speed than on a CPU. This is due to the lack of advanced caching features and processes existent on GPU hardware. Furthermore, the size of the DEM plays a greater role in execution speeds for the CPU

Table 3
Performance test results.

Execution time (s)										
Datasets	Test points	Algorithms								
		CPU-DDA	CPU-R3	CPU-R2	CPU-XDRAW	GPU DDA	GPU R3	GPU R2	GPU XDRAW	XDraw-O
DEM500	TP1	0.080	0.270	0.170	0.070	0.026	0.026	0.058	0.040	0.012
	TP2	0.080	0.300	0.180	0.090	0.022	0.022	0.052	0.063	0.014
	TP3	0.080	0.280	0.180	0.080	0.021	0.023	0.048	0.036	0.015
DEM300	TP1	0.240	0.740	0.480	0.210	0.030	0.033	0.089	0.050	0.025
	TP2	0.230	0.770	0.480	0.220	0.030	0.069	0.080	0.052	0.023
	TP3	0.220	0.750	0.580	0.220	0.031	0.074	0.076	0.095	0.021
DEM250	TP1	0.540	1.640	1.100	0.510	0.063	0.099	0.161	0.133	0.030
	TP2	0.620	1.990	1.170	0.560	0.063	0.105	0.150	0.132	0.028
	TP3	0.550	1.820	1.130	0.500	0.066	0.098	0.138	0.108	0.026
DEM200	TP1	0.910	3.040	2.080	0.820	0.109	0.147	0.240	0.183	0.042
	TP2	0.990	3.120	2.150	0.880	0.109	0.141	0.234	0.206	0.040
	TP3	0.930	3.010	2.010	0.870	0.123	0.144	0.233	0.183	0.036
DEM150	TP1	1.940	5.220	3.950	1.630	0.185	0.211	0.316	0.231	0.089
	TP2	1.990	5.120	4.090	1.650	0.191	0.215	0.310	0.258	0.083
	TP3	1.940	5.200	3.950	1.620	0.192	0.216	0.302	0.230	0.078
Max		1.990	5.220	4.090	1.650	0.192	0.216	0.316	0.258	0.089
Min		0.080	0.270	0.170	0.070	0.021	0.022	0.048	0.036	0.012
Standard deviation		0.694	1.825	1.420	0.574	0.064	0.069	0.099	0.077	0.025
Average		0.756	2.218	1.580	0.662	0.084	0.108	0.166	0.133	0.038

Table 4
Reductions in execution time.

	Algorithm				
	DDA	R3	R2	XDraw	XDraw-O
CPU	0.756	2.218	1.58	0.662	–
GPU	0.084	0.1081	0.1658	0.133	0.0375
Execution time reduction	88.89%	95.13%	89.51%	79.91%	–

algorithms than the GPU algorithms. It is hypothesized that since the DEM sizes are not linearly related, the relationship between the data I/O operations and the data computations are different between the two hardware types. The GPU may be capable of retrieving the data faster than the CPU from its working memory and further tests may confirm whether this is the case.

For the GPU to GPU algorithm comparison, several results are of note. XDraw did not perform as fast as either DDA with an average of 0.084 or R3 with an average of 0.108. In most instances XDraw was second only to R2 in terms of lowest performance across the dataset, with an average of 0.166 s. This may be due to the fact that XDraw requires a non-standard data access pattern when compared to the line of sight algorithms. As XDraw did not contain any explicit memory management features, it was up to the C++ AMP compiler to determine appropriate buffer sizes for the SMs. The results detailed in Table 3 may reflect the inadequacy of the compiler to choose appropriate data sizes and operations automatically. This further supports the use of XDraw-O, which contains more fine-tuned memory management techniques. A further result of this testing process revealed that DDA performed better than R3, R2 and XDraw in most instances. As DDA was not the focus of this research, instead it was used as a benchmark for XDraw, this result is revealing. Future research may look into further optimizations of DDA to determine if the speed can be further increased. This effort may follow the research of Ferreira [4] in an attempt to optimize Van Kreveld's algorithm or the DDA algorithm of Zhao, Padmanabhan and Wang [16].

XDraw-O can be seen to possess a faster execution time overall when compared to the other four GPU algorithms as well as being faster than all CPU viewshed analysis algorithms. It can be deduced from the algorithmic performance of XDraw-O listed in Table 3 that XDraw responded well to the tiling and branching code detailed in the previous section. A possible explanation for this will follow. First, the static buffer tiles can be assumed to be more efficient

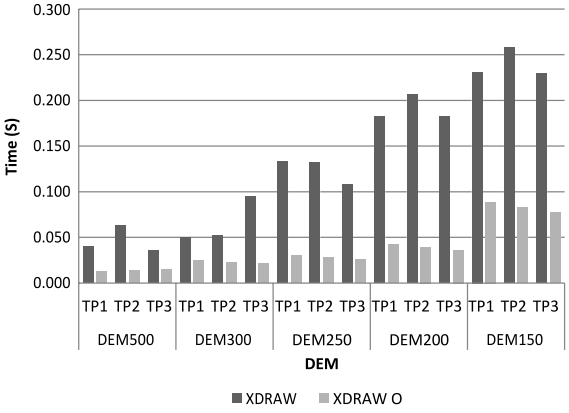


Fig. 3. XDraw and XDraw-O performance comparison.

than those of base XDraw. Specifically, as data is accessed from global memory in large contiguous chunks at the beginning of the kernel initialization, stalls are not as frequent or detrimental to the algorithm's performance. Note however that C++ AMP performs automatic tile management when no tile size is explicitly specified; these sizes may not be optimal for the size or data access pattern of the array in GPU memory. Second, the execution of each thread as part of an octant does not have to resolve a code flow branch, whereby both paths are executed and one is discarded. The code now calculates whether the elevation is the maximum via a simple standard math library max operation. Both of these changes can be seen to have a positive change on the algorithm's performance and Fig. 3 displays this trend.

Fig. 5 describes the sampling density with respect to the algorithm's execution time. It can be observed that XDraw-O responds less severely to the density of a DEM with respect to execution time when compared to the GPU XDraw algorithm for

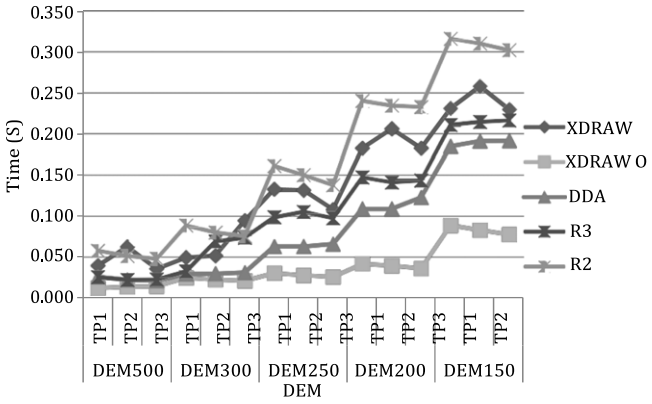


Fig. 4. GPU algorithm performance comparison.

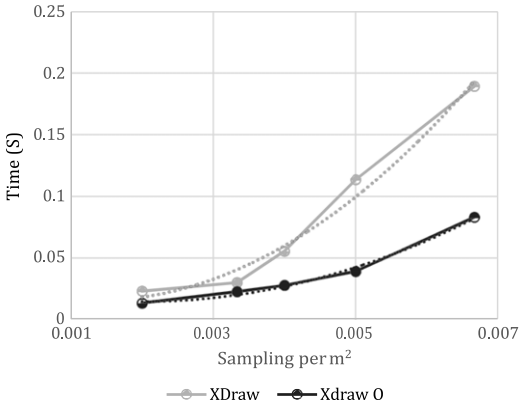


Fig. 5. Sampling density with respect to execution time.

terrains of increasing density. Even higher densities will need to be tested to determine when the trend ends. As described previously, XDraw-O differs structurally from XDraw in terms of kernel anatomy. It can be thus argued that from the positive performance characteristics exhibited by XDraw-O benefited from this structure. To possibly explain the implications of this, it can be argued that the memory contiguity of using eight separate kernels is of less importance than the larger thread throughput of the two-kernel model used in XDraw-O. To generalize further, it can then be argued that XDraw responds better to improvements in bandwidth throughput than in direct memory access patterns.

Overall, it can be gathered from the data presented that XDraw-O produces a viewshed on average 72.2% faster than that of Xdraw as demonstrated in Figs. 3 and 4. More importantly, XDraw-O also performed faster than DDA, R3 and R2 in all cases. It can be then stated that XDraw-O is the most efficient algorithm, across the test cases for generating viewsheds in the general case. Further analysis will be required to determine if this performance difference is maintained when memory optimization techniques are applied to the LoS algorithms, such as those proposed by Zhao, Padmanabhan and Wang [16] for the DDA algorithm or the work of Xia, Yang and Xingmin [14] for creating more optimized ray traversals across a DEM. It can be assumed that the number of GPU cores would significantly affect the performance of the algorithm. Increasing the number of cores would be a viable future point of research in determining the exact bottlenecks with respect to performance. The performance results detailed correlate well with the performance increases of many previous GPU viewshed studies. Xia, Yang and Xingmin [14] and Zhao, Padmanabhan and Wang [16] demonstrated that significant performance improvements can be made with careful consideration to data operations. Narasiman et al. [9] also demonstrated that spatial independency is a key factor when

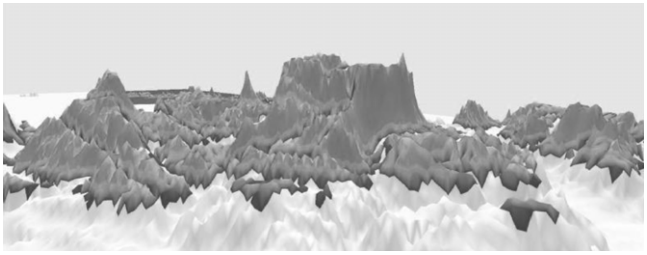


Fig. 6. Medium range visible terrain.

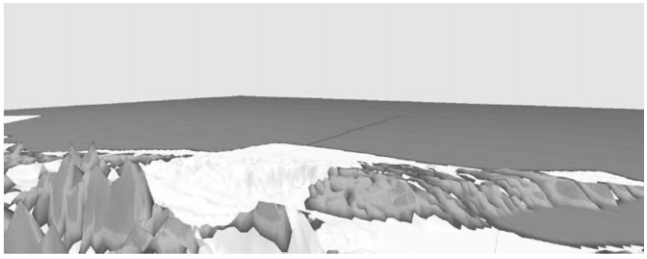


Fig. 7. Long range visible terrain.

generating viewsheds on a GPU. As XDraw contains excellent spatial independency, the results in this paper support this hypothesis.

The algorithm produced moderately accurate viewsheds over the course of CPU, GPU and Optimized GPU simulations. The algorithm suffered from several visual artifacts when rendered on the GPU. This may be due to an incorrect tile edge-case. The issue was not explored fully but would be corrected in subsequent versions of the GPU algorithm. Figs. 6 and 7 display the visual results of the Xdraw-O algorithm from Test Point 1 taken from different angles near the observer point to demonstrate the visible and invisible portions of terrain.

4. Conclusion and further work

This paper has presented a novel rendition of the XDraw visibility algorithm, which executes in a GPU context. The new algorithm performs well in the test cases described and sees an improvement in speed when compared to all algorithms to a varying extent. Several avenues of future research are possible from this point. XDraw-O can be seen as the first step in future efforts to parallelize viewshed analysis. Further optimizations to XDraw generally may include a variety techniques, such as multiple GPU solutions, unified CPU/GPU architecture and the use of more advance kernel dispatch processes. The algorithm will significantly increase the performance of some viewshed analysis scenarios suffered from several edge-case accuracy issues. These issues will need to be rectified in the future. More broadly, GPU techniques such as orthographic rendering of terrain as a 3D environment may prove even more beneficial to viewshed analysis speeds.

Acknowledgment

This research was conducted with the assistance from Myriax.

References

[1] F. Chao, Y. Chongjun, C. Zhuo, Y. Xiaojing, G. Hantao, Parallel algorithm for viewshed analysis on a modern GPU, Int. J. Digital Earth 4 (6) (2011) 471–486.
[2] M. De Smith, M. Goodchild, P. Longley, Geospatial Analysis: A Comprehensive Guide to Principles, Techniques and Software Tools, 2007.
[3] W. Feng, P. Deji, L. Yuan, Y. Liuzhong, W. Hongbo, A parallel algorithm for viewshed analysis in three-dimensional digital earth, Comput. Geosci. (2014).
[4] C. Ferreira, et al., A parallel sweep line algorithm for visibility computation, in: GeoInfo, 2013.

[5] W. Franklin, S. Mehta, Geometric algorithms for siting of air defense missile batteries, A Research Project for Battle, No. 2756, 1994.
[6] Y. Gao, H. Yu, Y. Liu, Y. Liu, M. Liu, Y. Zhao, Optimization for viewshed analysis on GPU, in: 2011 19th International Conference on Geoinformatics, IEEE, 2011, pp. 1–5.
[7] B. Kaučič, B. Zalik, Comparison of viewshed algorithms on regular spaced points, in: Proceedings of the 18th spring conference on computer graphics, 2002, pp. 177–183.
[8] Myriax, Myriax EonFusion: Easy data access, visualisation and fusion, viewed 5/10 2014, 2013. http://www.eonfusion.com/product/13400007.
[9] V. Narasiman, M. Shebanow, C. Lee, R. Miftakhutdinov, O. Mutlu, Y. Patt, Improving GPU performance via large warps and two-level warp scheduling, in: Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture, 2011.
[10] J. Owens, M. Houston, D. Luebke, S. Green, J. Stone, J. Phillips, GPU computing, Proc. IEEE 96 (5) (2008) 879–899.
[11] N. Stojanovic, D. Stojanovic, 2013 11th International Conference on Telecommunications in Modern Satellite, Cable & Broadcasting Services, TELSIKS, Vol. 2, 2013, pp. 397–400.
[12] D. Strnad, Parallel terrain visibility calculation on the graphics processing unit, Concurr. Comput.: Pract. Exp. 23 (18) (2011) 2452–2462.
[13] J. Wang, G. Robinson, K. White, A fast solution to local viewshed computation using grid-based digital elevation models, Photogramm. Eng. Remote Sens. 62 (10) (1996) 1157–1164.
[14] Y. Xia, K. Li, L. Xiu-me, Accelerating geospatial analysis on GPUs using CUDA, J. Zhejiang Univ. Sci. C 12 (12) (2011) 990–999;
Y. Xia, L. Yang, S. Xingmin, Parallel viewshed analysis on GPU using CUDA, in: 2010 Third International Joint Conference on Computational Science and Optimization (CSO), Vol. 1, IEEE, 2010, pp. 373–374.
[15] Y. Zhao, B. Chen, Y. Fang, Z. Huang, Y. Liu, H. Yu, A parallel implementation of nearest neighbor analysis based on GPGPU, in: 2011 19th International Conference on Geoinformatics, IEEE, 2011, pp. 1–6.
[16] Y. Zhao, A. Padmanabhan, S. Wang, A parallel computing approach to viewshed analysis of large terrain data using graphics processing units, Int. J. Geogr. Inf. Sci. 27 (2) (2012) 363–384.

Glossary

DDA*:* Digital Differential Analyzer. A Line of Sight algorithm for calculating visibility using integer arithmetic.
DEM*:* Digital Elevation Model. A digital representation of topography.
GPGPU*:* General Purpose Graphics Processing Unit. A Graphics Processing Unit intended to perform calculations for a program whose output is not strictly a rendered image.
LoS*:* Line of Sight. The binary representation of visibility from one point to another.
R2*:* A Line of Sight visibility algorithm similar to R3 but contains more assumptions concerning approximate visibility.
R3*:* A Line of Sight visibility algorithm. The algorithm iterates through every point of a DEM to determine visibility and can be considered the most accurate of the cohort.
SM*:* Stream multi-processor. A concurrent processor which executes kernel threads in parallel.
XDraw*:* A visibility analysis algorithm which compounds the results of visibility from previous rings of growth into the next row.



Aran J. Cauchi-Saunders is a Ph.D. Candidate at the School of Engineering and ICT at the University of Tasmania, Australia. He completed his Bachelor of Computing (Hons. First Class) in 2013, producing a thesis entitled 'Experiments in Parallel Viewshed Analysis', focusing on GPU techniques to increase the performance and accuracy of viewshed analysis.



Ian J. Lewis is a Lecturer at the School of Engineering and ICT, University of Tasmania in Australia. Ian primarily teaches and researches in the domains of computer and video games, artificial intelligence, and programming. Ian is primarily interested in the effects of video games on players and the potential to harness their positive effects for other purposes.